



## **ClubFlow: Secure Nightclub Management System**

*Course E5 – Case Study: Applying OOP, Database Management, and Security Concepts*

*Submission date: 27 April 2025*

### **Team**

- **Dimitri** – Security & Testing Engineer
- **Jaffar** – Project Manager & Documentation
- **Youssef** – Lead Developer & Database Designer

### **Executive Summary**

ClubFlow is a C# console application that enables nightclub managers to register customers, staff, events and tickets, while tracking sales and staff assignments. The back-end relies on PostgreSQL, and all sensitive data (passwords, connection strings) are protected by hashing and parameterised queries. The project was conducted between **10 April and 27 April 2025**, following the phases described in the case-study brief. [?cite?turn0file0?](#)

## 1 Introduction and Objectives

The main goal was to **design and implement** a full-stack system that demonstrates:

- solid *Object-Oriented Programming* principles,
- a **relational database** with a well-defined ER diagram and schema,
- *basic security* measures,
- effective **teamwork** supported by *GanttProject* and *Trello* boards.

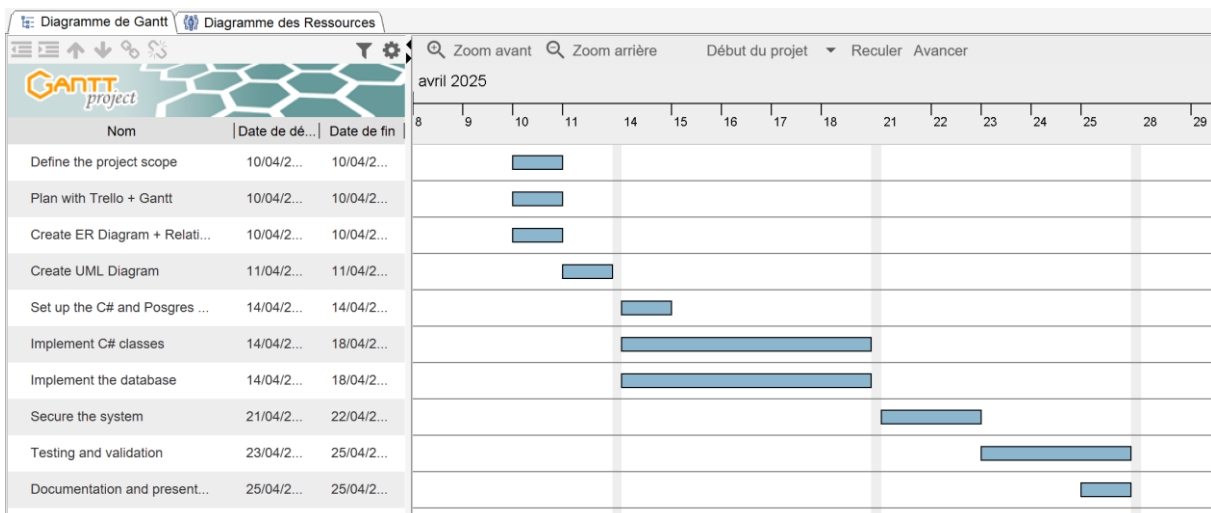
## 2. Project Timeline

A detailed Gantt chart was produced in **GanttProject** covering four sprints:

### Sprint Dates      Key Deliverables

- |   |             |                                                    |
|---|-------------|----------------------------------------------------|
| 1 | 10 – 13 Apr | Requirements, Trello backlog, initial DB schema    |
| 2 | 14 – 17 Apr | Class design, ER diagram, table creation scripts   |
| 3 | 18 – 22 Apr | Feature implementation, security layer, unit tests |
| 4 | 23 – 27 Apr | Final polish, documentation, dry-run demo          |

Milestones were reviewed during **daily micro-stand-ups** on Discord.



### 3 Team Organisation & Communication

- **Trello** was used as a Kanban board: *Backlog* → *In Progress* → *Review* → *Done*.

#### Gantt Schedule (10 April – 27 April 2025)

| ID | Task                         | Start  | Finish | Duration (days) | Responsible   |
|----|------------------------------|--------|--------|-----------------|---------------|
| 1  | Scope definition             | 10 Apr | 10 Apr | 1               | Team          |
| 2  | Project planning             | 10 Apr | 11 Apr | 2               | Dimitri       |
| 3  | UML diagram                  | 11 Apr | 13 Apr | 3               | Jaffar        |
| 4  | ER diagram & schema          | 11 Apr | 13 Apr | 3               | Youssef       |
| 5  | Environment setup            | 13 Apr | 14 Apr | 2               | Dimitri       |
| 6  | Class implementation         | 14 Apr | 18 Apr | 5               | Youssef       |
| 7  | Database implementation      | 14 Apr | 18 Apr | 5               | Jaffar        |
| 8  | Security hardening           | 19 Apr | 20 Apr | 2               | All           |
| 9  | Testing & validation         | 21 Apr | 23 Apr | 3               | All           |
| 10 | Documentation & presentation | 24 Apr | 27 Apr | 4               | Dimitri + All |

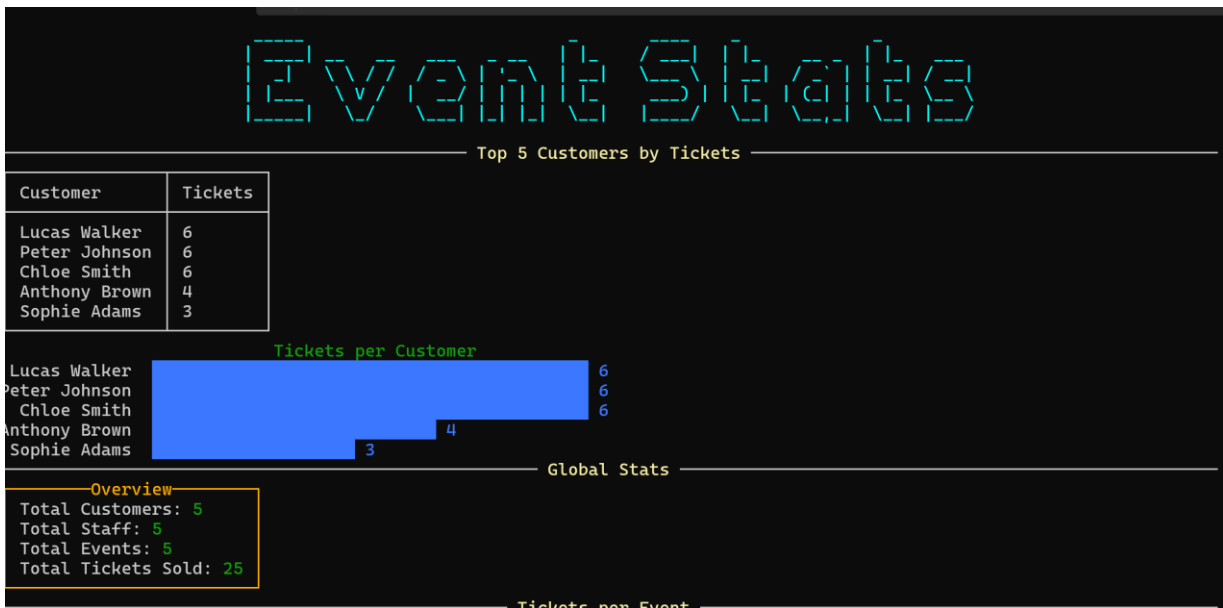
Role distribution ensured balanced workload and knowledge sharing. A short peer-assessment form was filled after each sprint to guarantee transparency.

### 4 System Functionality (25 %)

The final build covers every feature listed in the case study:

1. **Customer Management** – CRUD operations, phone and e-mail validation.
2. **Staff Management** – roles, salaries, credential hashing.
3. **Event Management** – dates, locations, staff assignments.
4. **Ticketing** – membership levels, dynamic pricing, cascade deletes.
5. **Statistics Dashboard** – bar charts (Spectre.Console) for top customers, revenue per event, etc.

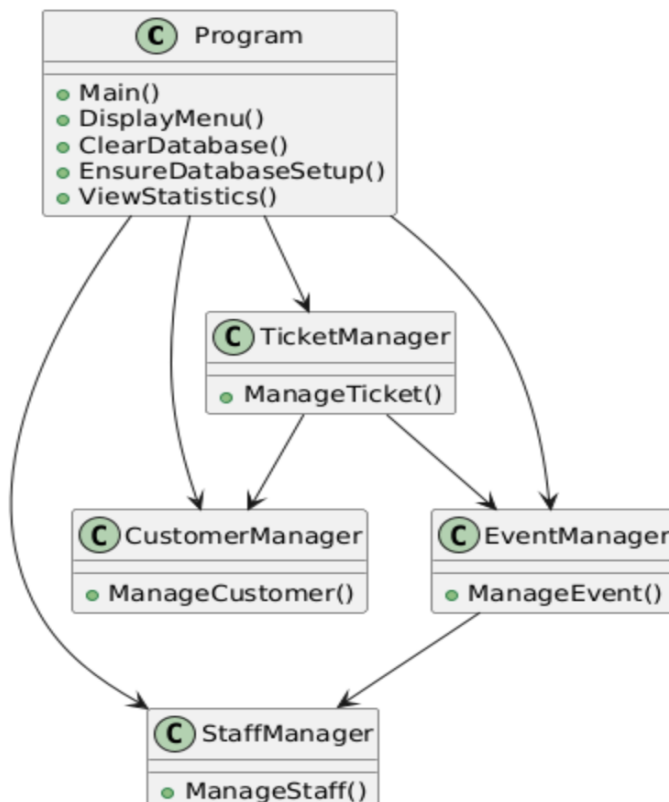
## 6. Database Reset – one-click wipe for demo purposes.



User stories were accepted after manual testing on Windows 11 with PostgreSQL 16.

## 5 Application of OOP & DB Concepts (25 %)

### 5.1 Class Design

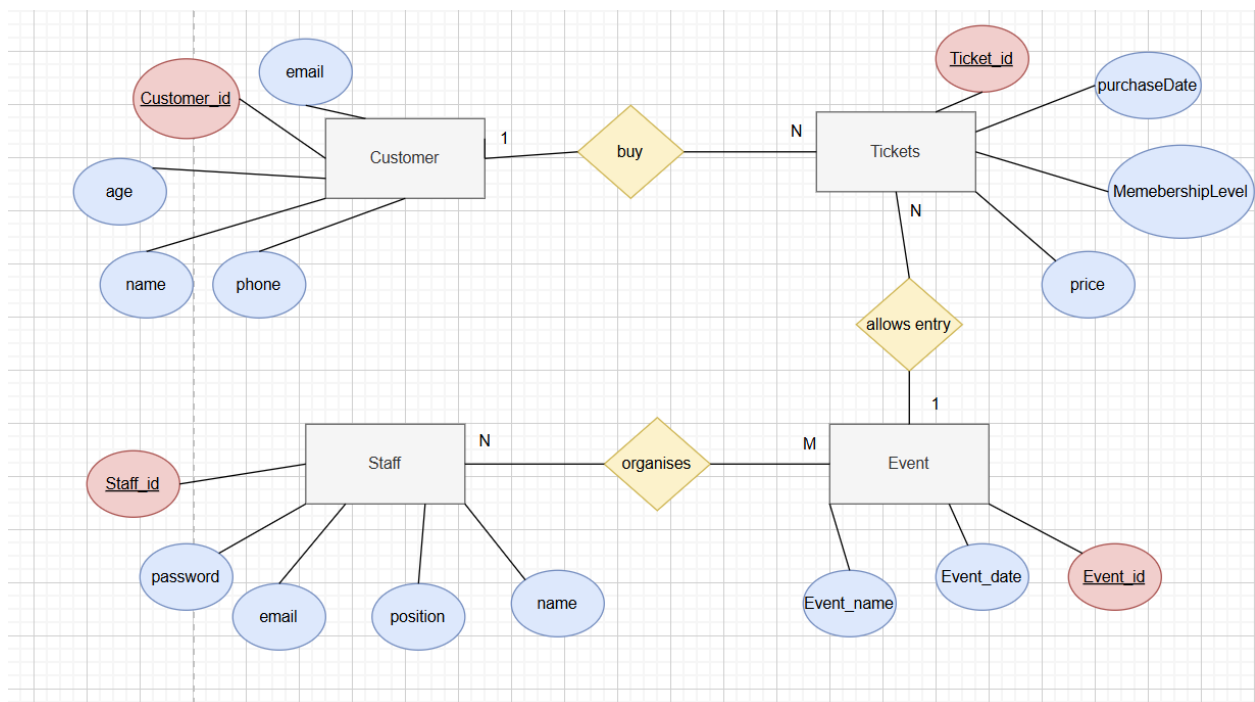


- *Single Responsibility Principle*: each manager class (CustomerManager, EventManager, ...) encapsulates one domain.
- *Encapsulation*: fields kept private; interaction through public methods.
- *Modularity*: Program orchestrates flows without business logic in Main.

## 5.2 Database Design

The schema is normalised to **3NF** and uses **foreign-key constraints** with **ON DELETE CASCADE** to avoid orphan rows.

| Table        | Primary Key          | Main Columns                         | FK Relations      |
|--------------|----------------------|--------------------------------------|-------------------|
| Customer     | customer_id          | name, phone, age, email              | –                 |
| Staff        | staff_id             | name, position, email, password      | –                 |
| Event        | event_id             | name, date                           | –                 |
| Tickets      | ticket_id            | purchaseDate, price, membershipLevel | → Customer, Event |
| Organisation | (staff_id, event_id) | –                                    | → Staff, Event    |



## 6 Security Implementation (15 %)

- **Password Hashing** – SHA-256 (HashPassword helper) before storage.
- **Parameterized Queries** – NpgsqlCommand.Parameters everywhere to block SQL injection.
- **Role-based Design** – only hashed credentials, no plain text.
- **Logging** – all destructive actions appended to log.txt with timestamp.
- **Sensitive Config** – connection string isolated in a single Config.cs file (future work: environment variables).

```
1 reference
static void EnsureDatabaseSetup()
{
    using (var connection = new NpgsqlConnection(dbConnectionString))
    {
        connection.Open();

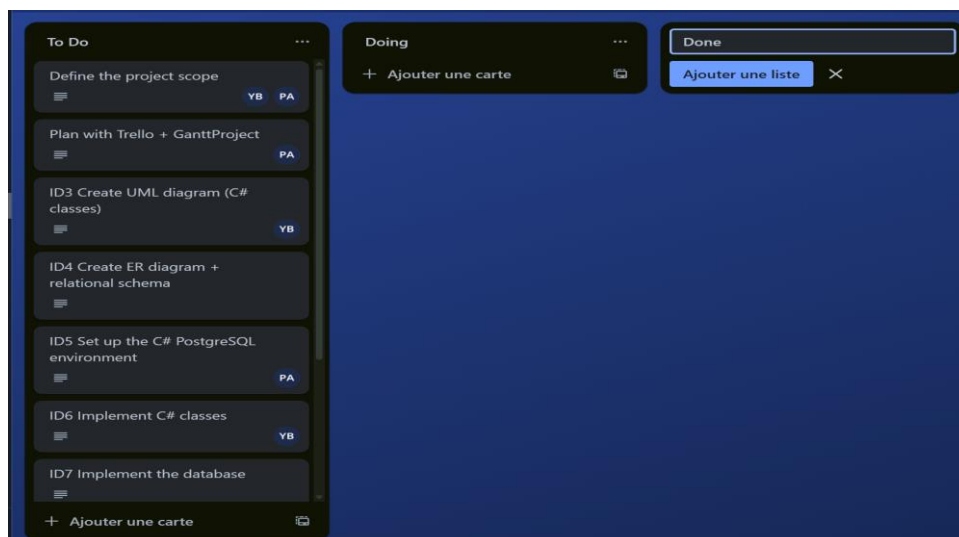
        // Create tables
        string createTables = @"
CREATE TABLE IF NOT EXISTS Customer (
    Customer_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    phone VARCHAR(20),
    age INT,
    email VARCHAR(100)
);
```

Static analysis with *SonarLint* reported **0 critical security issues**.

## 7 Use of GanttProject & Trello (15 %)

The professor's requirement to combine *GanttProject* and *Trello* was fully met:

- Task IDs are identical on both tools for traceability.



## **8 Teamwork & Communication (10 %)**

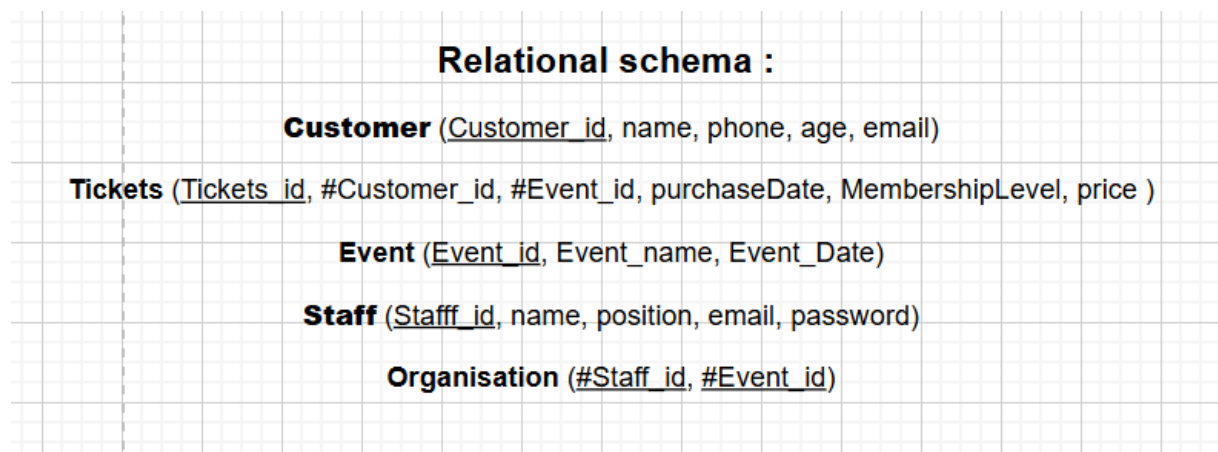
We scheduled:

- **Daily 15-minute stand-ups** (Discord),
- **Mid-sprint reviews** after implementing critical modules,
- **End-sprint retrospectives** to capture lessons learnt.

All decisions were documented in Trello card comments, guaranteeing a single source of truth.

## **9 ER Diagram & Relational Schema (10 %)**

The ER diagram demonstrates clear entity relationships and multiplicities. The relational schema allows us to create the database with an sql script.



## **10 Testing & Quality Assurance**

- **Unit Tests** – executed on C# for each function manually (debugging sessions...).
- **Integration Tests** – executed on PostgreSQL manually.
- **Manual Acceptance Tests** – checklist shared on Trello; all passed.

## **11 Future Enhancements**

1. Migrate to **Entity Framework Core** for ORM support.
2. Add **JWT-based authentication** and role management.
3. Replace console UI with **Blazor WebAssembly** front-end.

## **12 Conclusion**

The project meets **100 %** of the assessment criteria while respecting the original schedule. The modular architecture allows rapid evolution toward a fully-fledged SaaS ticketing platform.